



FOR JAVA DEVELOPERS MOVING TO NESTJS

33 updates, sent again.

Spring Boot to NestJS, episode 31: server sent events

A stream needs a lifetime and a resume point. The client remembers where it was; the server still has to honor that cursor.

A real EventSource on a Spring Boot SSE stream for 70 s: the stream timed out every 30 s, the client reconnected by itself, and 33 ids arrived more than once.

The 30 s was Tomcat's async timeout default, which Spring Boot leaves in place. Nest imposed no timeout in the same test.

Neither stack honoured Last-Event-ID until the endpoint read it. Then both resumed where the client left off.

In scope

- SseEmitter and @Sse
- Stream lifetime
- Last-Event-ID and reconnects
- Cleanup and compression

Not in scope

- Proxy heartbeats (not measured)
- Slow consumers (see the WebSockets episode)

At the time of recording Tomcat's async timeout default is 30000 ms.

01 The measurements

SPRING, NO TIMEOUT ARGUMENT	ended after 31 s (Tomcat default)
NEST, @Sse	open after 70 s
RECONNECT, HEADER NOT READ	both restarted at 1
RECONNECT, HEADER READ	both resumed at 4
EVENTSOURCE, SPRING, 70 s	3 connections, 33 repeated
EVENTSOURCE, SPRING, HEADER READ	3 connections, 0 repeated
CLIENT LEAVES / COMPRESSION ON	same on both

One event a second, each with an id.



02 Honour the cursor

EventSource sends Last-Event-ID on every reconnect. Read it and continue from the next id; in a real system that means events you can read back by id.

```
payments.controller.ts
```

```
@Sse('payments')
payments(@Headers('last-event-id') lastEventId?: string): Observable<MessageEvent> {
  const start = lastEventId ? Number(lastEventId) + 1 : 1;
  return this.events.from(start); // your store, read by id
}
```

header read: both stacks resumed at 4; with EventSource, 0 repeated ids

Run on a real machine, 2026-10-01, by `scripts/verify-sse.sh` in the course repository.

03 Set the lifetime on purpose

In Spring, pass a timeout to the emitter or set `spring.mvc.async.request-timeout`. In Nest, end the Observable when the stream should end.

```
PaymentsStream.java
```

```
SseEmitter emitter = new SseEmitter(Duration.ofMinutes(30).toMillis());
```

```
application.properties
```

```
spring.mvc.async.request-timeout=30m
```

no timeout argument: the stream ended after 31 s, onTimeout fired

Run on a real machine, 2026-10-01, by `scripts/verify-sse.sh` in the course repository.

Before you ship an SSE endpoint

1. How long does a stream live, and who decided?
2. Does every event carry an id?
3. Does the endpoint read Last-Event-ID?
4. Can you read past events back by id?
5. Does a proxy close idle connections? Then send heartbeats.

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 X https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-sse.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS server sent events https://docs.nestjs.com/techniques/server-sent-events

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. Apache Tomcat is a trademark of the Apache Software Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.